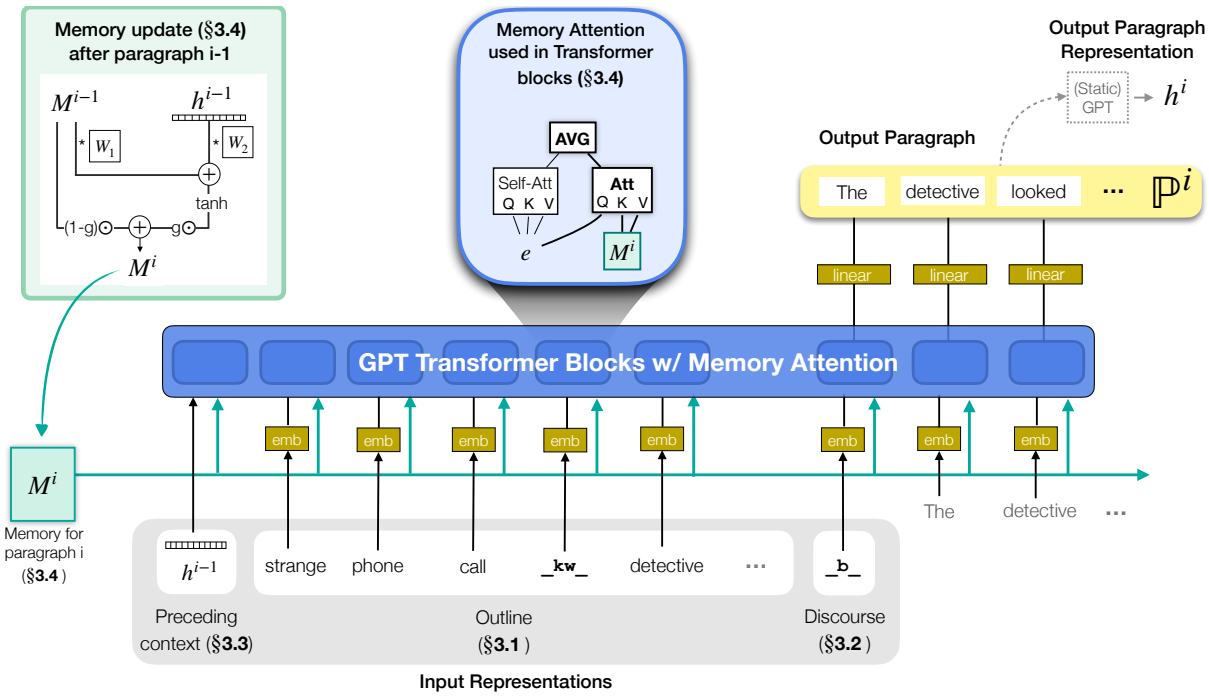

Figure 2: PLOTMACHINES: The model generates a paragraph $\mathbb{P}^i$ using the memory (M^{i-1}), the previous paragraph representation (h^{i-1}), the outline representation (o) and discourse representation (d^i). First, a gated update mechanism updates the memory using the previous memory and previous paragraph representation. Each transformer block includes extra attention over the current memory matrix M^i. The previous paragraph representation, h^{i-1}, the outline, and discourse tag (e.g. _b_) are also prepended to the generation as an input sequence (gray box). The output tokens of the generated paragraph are used to compute h^i using a static GPT model.

generated. We use special **_kw_** tokens to delimit each plot point in the outline and end the sequence with a special **_endkw_** token. We truncate the entire outline to maximum of n tokens. For example, an outline containing two plot points ({'strange phone call', 'detective'}) is turned into the input sequence:

`strange phone call` **_kw_** `detective` **_endkw_**

3.2 Discourse Representation

We posit that there are stylistic differences in how the beginning, middle and end of a story are written. To learn these differences, we introduce d^i, discourse information about whether the i-th paragraph is an introduction, body, or conclusion paragraph. We append a special token to the outline representation as part of the input sequence: **_i_**, **_b_**, **_c_** for the introduction, body, and conclusion paragraphs respectively[8].

3.3 Preceding Context Representation

With the goal of incorporating previous story context in generating each paragraph, we use h^{i-1}, an embedded representation of the previous paragraph, which is added to the model input. More concretely, h^{i-1} is computed as the average embedding of GPT output representations of words from the previous paragraph (using a GPT model that is static, i.e. not finetuned). The h^{i-1} vector is used as an initial input to the transformer architecture, as shown in Figure 2.

3.4 Memory Representation

We implement memory to address two key task challenges. First, we want to keep track of the portions of the outline that have been mentioned. Second, we want to maintain semantic coherence across the entire story. To address these two challenges, we implement the memory as consisting of two parts: K, a set of vectors keeping track of outline points, and D, a matrix that stores a latent topic distribution of what's been written so far.

Notation: We define d as the embedding size of the transformer model and n as the maximum number of tokens in the outline. Memory is treated as a $\mathbb{R}^{d \times 2n}$ matrix which consists of two smaller matrices stacked together ($M = [K; D]$). K is a $\mathbb{R}^{d \times n}$ representation of outline points and D is a $\mathbb{R}^{d \times n}$ representation of the latent document state. K is

[8] We make the simplifying assumption that the first paragraph is an introduction, the last paragraph is the conclusion paragraph, and the other paragraphs are all body paragraphs.

initialized with embeddings representing each of the tokens in the outline and D is randomly initialized. The j-th column of memory at the timestep for paragraph i will be denoted M_j^i.

Updating memory: The memory is updated (top left corner of Fig. 2) using h^{i-1}, the average GPT output representation of the previous paragraph. We use update equations based on those in entity-based models such as Henaff et al. (2017). We use a gating mechanism, g, to allow the model to learn to flexibly control how much each cell in memory is updated, as below:

$$\hat{M}_j^i = tanh(W_1 M_j^{i-1} + W_2 h^{i-1}) \quad (2)$$

$$g_j^i = sigm(W_3 M_j^{i-1} + W_4 h^{i-1}) \quad (3)$$

$$M_j^i = (1 - g_j^i) \odot M_j^{i-1} + g_j^i \odot \hat{M}_j^i \quad (4)$$

where all W's are matrices of dimension $\mathbb{R}^{d \times d}$.

Transformer Blocks with Memory: Lastly, we must alter the GPT transformer blocks to include the memory in the language modeling. We alter the attention used within the transformer blocks to contain two parallel attention modules, as shown in Figure 2. One attention module (on the left in the figure) performs the standard GPT self-attention using transformer inputs to create queries, keys, and values. The other attention module uses transformer input to attend over the memory vectors (i.e., using the memory for creating key and value vectors). The outputs of both attention modules are averaged[9] before performing the remaining transformer block operations.

3.5 Training and Decoding

At training time, the model is trained end-to-end on the cross-entropy loss of predicting each paragraph. Gold representations of previous paragraphs in the story are used to update the memory and compute h^{i-1}. At decoding time, the model must decode a document starting with the first paragraph and use its own predictions to compute h^{i-1} and update the memory. Additionally, at decoding time, we assume a five paragraph structure (introduction, three body paragraphs, and conclusion) as a pre-set discourse structure to decode from.

4 Experiments

We present experiments comparing PLOTMACHINES with competitive baselines and ablations using automatic metrics and human judgements targeting multiple aspects of performance. In Sec. A.3, we also include example generations.

4.1 Experimental Set-up

Baselines: We compare with two models that have been used in related conditional story generation tasks. First, we train a Fusion model, from the original WritingPrompts dataset paper (Fan et al., 2018), using delimited outlines as a single input in place of a prompt. We also compare with the static storyline-to-story variant of Plan-and-Write (P&W-Static) from Yao et al. (2019), which is an LSTM-based model that we train by using the plot outline as delimited input.

Additionally, given the recent successes in text generation using large pre-trained LM's, we compare with these models, as well. We finetune the large-scale GROVER (Zellers et al., 2019) (equivalent to GPT-2 medium, 345M param) , which is a transformer-based language model that has been pre-trained for controllable text generation. To fine-tune GROVER, we give the outline as a delimited form of metadata. GROVER (345M param) has significantly larger capacity than PLOTMACHINES (160M param). Therefore, for more direct comparison, we also investigate a 460M parameter version of PLOTMACHINES that is built on top of GPT-2 medium (Radford et al., 2019) instead of GPT.

Unlike our models, the baselines are trained with the traditional generation framework, to generate an entire document conditioned on outlines without generating each paragraph recurrently.

Ablated PLOTMACHINES Models: We also show results in Table 2 on ablated versions of our model. First, we use the base GPT and GPT2 models, that are fine-tuned similarly to our model but using only outline inputs (without memory, preceding context, or discourse representations). Second, we investigate the effects of using the preceding context representation but still excluding memory and discourse tokens (**PM-NOMEM-NODISC**). Lastly, we use **PM-NOMEM**, a model variant that excludes the memory but uses outline, discourse, and preceding context representations as input.

Details: We use the HuggingFace implementations of GPT and GPT-2, and we fine-tune using ADAM. For generating with our models, we use nucleus sampling with repetition penalties (Holtzman et al., 2019; Keskar et al., 2019) using $p = 90$ and $\theta = 1.5$ for GPT and $p = 70$ and $\theta = 1.4$ for

[9] We experimented with a few other variants of implementing multiple attention mechanisms within the transformer blocks, but found this to be empirically effective.

	Wikiplots			WritingPrompts			New York Times		
Model	R-1	R-2	R-L	R-1	R-2	R-L	R-1	R-2	R-L
P&W-Static (Yao et al., 2019)	17.0	3.3	13.6	19.2	3.6	14.4	19.3	4.6	15.6
Fusion (Fan et al., 2018)	22.7	6.0	17.4	14.3	1.7	9.6	**23.2**	**7.2**	**18.1**
GROVER (Zellers et al., 2019)	19.6	5.9	12.5	23.7	5.3	17.2	20.0	5.8	14.2
PLOTMACHINES (GPT)	20.2	5.3	16.0	30.5	5.3	25.4	21.2	5.0	15.5
– base (GPT) (Radford et al., 2018)	13.2	2.0	7.9	22.1	2.7	14.3	13.9	1.6	8.3
PLOTMACHINES (GPT-2)	**22.8**	**6.5**	**17.5**	**31.1**	**6.7**	**26.1**	22.1	6.4	16.5
– PM-NOMEM (GPT-2)	20.5	4.9	15.5	26.6	3.7	23.5	20.0	5.4	14.4
– PM-NOMEM-NODISC (GPT-2)	19.3	1.7	13.9	26.8	4.5	23.2	18.4	3.4	14.2
– base (GPT-2) (Radford et al., 2019)	18.5	3.9	13.3	26.5	4.6	20.5	19.2	4.7	13.6

Table 2: ROUGE Results on Wiki, WritingPrompts and NYTimes Datasets. The top block represents the baseline models on story/article generation, while the bottom blocks include ablations of our PLOTMACHINES models.

GPT-2 (based on a hyperparameter sweep using grid search with cross-entropy on the dev. data). We use a minimum sequence length of 100 bpe tokens per paragraph and a maximum sequence length of 400, 922 bpe per paragraph for GPT and GPT-2, respectively. We set n, the maximum number of outline tokens and memory dimensions to 100. We used the settings for the baselines from their respective papers and codebases.

4.2 Automatic Metrics

In this section, we evaluate performance using different automatic metrics. We compute ROUGE scores (Lin, 2004) and self-BLEU (Zhu et al., 2018) following from previous work (Shen et al., 2019; Zhu et al., 2018) showing that a large ROUGE score together with a low self-BLEU score can demonstrate a model's ability to generate realistic-looking as well as diverse generations.

Coverage We compute ROUGE scores (Lin, 2004) with respect to the gold stories (Table 2). Results show that the full PLOTMACHINES achieves comparable or higher ROUGE on all three datasets. Both PLOTMACHINES variants (using GPT or GPT-2 as a base) achieve improvements over GROVER, even though GROVER includes significantly more parameters than the model using GPT.

Ablations In the bottom block of Table 2, we compare performance of ablated versions of PLOTMACHINES. First, we compare GPT-2 with PM-NOMEM-NODISC, which differs by including preceding context representations. We observe that PM-NOMEM-NODISC performs slightly better than GPT-2, emphasizing the importance of including context from the previous paragraph. Second, we investigate the impact of discourse structure representations. We compare PM-NOMEM-NODISC, which omits the discourse token, with PM-NOMEM, which uses the discourse token. As shown in Table 2, PM-NOMEM generally has higher ROUGE scores than PM-NOMEM-NODISC, indicating that the discourse representation is beneficial to the model. Lastly, we compare PM-NOMEM with the full PLOTMACHINES to determine the effects of having a memory component. Our full model with memory has large ROUGE score improvements over PM-NOMEM, underscoring the importance of the plot state tracking.

Diversity We evaluate the diversity of generated paragraphs from our models using self-BLEU scores (Zhu et al., 2018). In Table 3, we report the self-BLEU scores along with the average length of each generated story. Using all the generated documents from a model, we take one generated document as hypothesis and the others as reference, and calculate BLEU score for every generated document, and define the average BLEU score to be the self-BLEU of the model. While the Fusion model achieved relatively high ROUGE scores, it has generally worse diversity scores (much higher self-BLEU in Table 3). It may be that this model's high ROUGE scores were obtained by producing text that is more repetitive and generic.[10] In contrast, PLOTMACHINES generally achieves good performance on both ROUGE and diversity scores, with self-BLEU scores that are lower than most other models. Notably, they generally have more similar self-BLEU scores to the actual gold stories, indicating that the language diversity is more similar to what humans write.

4.3 Human Evaluations

Due to the limitations of automatic metrics, we also perform extensive human evaluations. We conduct human studies to explore how generated stories compare along three dimensions: outline

[10] We show an example output from the Fusion model in Figure 13.